

Padrões de projeto de arquitetura de sistemas

Apresentação

O processo de desenvolvimento de *software* consiste em inúmeras atividades determinadas e organizadas que visam a resolver um problema da área de tecnologia da informação. O objetivo principal dessas atividades é satisfazer as necessidades dos clientes/usuários do sistema. Para tanto, é preciso analisar os requisitos do sistema e projetar e implementar um código padronizado para evitar erros e falhas.

Padronizar consiste em empregar técnicas e padrões que auxiliam no desenvolvimento e aumentam a produtividade, a segurança e a qualidade no projeto. Um padrão é um molde, um exemplo, um protótipo, um paradigma ou uma referência de algo a seguir. Um projeto é um desenho, uma planta, um esquema, um plano ou um programa.

Nesta Unidade de Aprendizagem, você vai explorar a identificação de padrões de projeto de arquitetura de sistemas, destacando o projeto de *software* embasado nesses padrões. Você também vai compreender a relevância de fornecer exemplos concretos desses padrões, enriquecendo a compreensão prática de suas aplicações. Além disso, vai entender como esses conceitos se integram e contribuem para uma visão abrangente e aplicada da arquitetura de sistemas.

Bons estudos.

Ao final desta Unidade de Aprendizagem, você deve apresentar os seguintes aprendizados:

- Identificar padrões de projeto de arquitetura de sistemas.
- Explicar o projeto de *software* baseado em padrões.
- Exemplificar padrões de projeto.

Infográfico

Encontrar o padrão de projeto ideal para resolver um problema de *software* nem sempre é uma tarefa fácil. Com uma variedade de repositórios de padrões disponíveis, como o GoF e o GRASP, cada um apresentando uma extensa lista de opções, é fundamental seguir critérios e avaliar informações que diferenciem e justifiquem o uso de cada padrão. O GoF, por exemplo, conta com 23 padrões distintos, tornando a escolha ainda mais desafiadora.

Neste Infográfico, você vai visualizar, compreender e analisar as comparações e a aplicabilidade da estrutura que envolve um padrão de projeto específico.

A imagem a seguir possui audiodescrição. Para acessar o recurso,

[clique aqui](#)

O QUE DESCREVE UM PADRÃO DE PROJETO?

Cada padrão de projeto é dividido em seções, estruturas que englobam informações e introduzem o padrão de projeto, possibilitando comparações, entendimento e aplicabilidade. Elas auxiliam e justificam o uso do padrão escolhido.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

1

NOME E CLASSIFICAÇÃO DO PROJETO

Expressa, de forma sucinta, a essência do projeto.

2

INTENÇÃO E OBJETIVO

Responde as perguntas: O que faz o padrão? Quais os seus princípios e sua intenção? Trata qual tópico ou problema?

3

OUTRO NOME REPRESENTADO

Outros nomes para reconhecer o padrão, caso existam.

4

MOTIVAÇÃO

Cenário que ilustra um problema e como o padrão de projeto pode solucioná-lo.

5

APLICABILIDADE

Onde o padrão pode ser aplicado? Que problemas ele pode resolver?

6

ESTRUTURA

Representação gráfica das classes do padrão usando uma notação baseada em UML (Unified Modeling Language).

7

PARTICIPAÇÃO E COLABORAÇÃO

Classes ou objetos que participam do padrão de projeto e suas responsabilidades, bem como as classes que colaboram para executá-las.

8

CONSEQUÊNCIAS

Como o padrão suporta a realização dos objetivos? Quais os custos e benefícios? Que aspecto estrutural ele permite variar independentemente?

9

IMPLEMENTAÇÃO

Sugestões técnicas para conhecer o padrão.

10

EXEMPLO DE CÓDIGO E USOS CONHECIDOS

Fragmentos ou blocos de códigos que ilustram o padrão de projeto. Exemplos do padrão em sistemas reais.

11

PADRÕES RELACIONADOS

Qual padrão está relacionado ao escolhido? Quais as diferenças importantes? Quais padrões complementam o padrão escolhido?

Ao explorar os elementos essenciais de um padrão de projeto, é possível compreender sua importância no desenvolvimento de *software*. Esses padrões fornecem diretrizes comprovadas e soluções para problemas recorrentes, permitindo a criação de sistemas mais eficientes, flexíveis e de qualidade. Por meio do estudo e da aplicação desses padrões, os desenvolvedores podem aprimorar suas habilidades, melhorar a estrutura e a arquitetura de seus projetos e criar soluções mais robustas e escaláveis.

Conteúdo do Livro

No universo do desenvolvimento de *software*, os *design patterns*, ou padrões de projeto, desempenham um papel fundamental. Esses padrões são referências aplicadas ao projeto que trazem soluções para problemas específicos encontrados durante o processo de desenvolvimento. Pode-se enxergá-los como soluções já testadas para determinados problemas, oferecendo uma base sólida para a criação de *software*. Conhecer e dominar os padrões de projeto possibilita o desenvolvimento de soluções cada vez melhores e mais eficientes, além de promover a reutilização de soluções já estabelecidas.

No capítulo **Padrões de projeto de arquitetura de sistemas**, base teórica desta Unidade de Aprendizagem, você vai aprofundar seus conhecimentos sobre os conceitos associados aos padrões de projeto e sobre as vantagens da utilização dos padrões de projeto, bem como entender a importância de escolher o padrão mais adequado para seu *software* orientado a objetos. Além disso, vai saber diferenciar os tipos de padrões existentes e compreender sua aplicabilidade em diversos contextos.

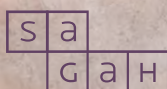
Boa leitura.

Os elementos gráficos deste capítulo possuem audiodescrição. Para acessar o recurso,

[clique aqui](#)

ARQUITETURA DE SISTEMAS

Aline Maciel Zenker



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Padrões de projeto de arquitetura de sistemas

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar padrões de projeto de arquiteturas de sistemas.
- Explicar o projeto de *software* baseado em padrões.
- Exemplificar padrões de projeto.

Introdução

O processo de desenvolvimento de *software* consiste em inúmeras atividades determinadas e organizadas, que têm como objetivo resolver um problema da área de tecnologia da informação. O objetivo principal dessas atividades é satisfazer as necessidades dos clientes/usuários do sistema. Para tanto, é preciso analisar os requisitos do sistema e projetar e implementar um código padronizado, para evitar erros e falhas.

Padronizar consiste em empregar técnicas e padrões que auxiliam no desenvolvimento e aumentam a produtividade, a segurança e a qualidade no projeto. Um padrão é um molde, um exemplo, um protótipo, um paradigma ou uma referência de algo a seguir. Um projeto é um desenho, uma planta, um esquema, um plano ou um programa.

Padrões de projeto (do inglês *design patterns*) são alternativas ou modelos de soluções para problemas encontrados no desenvolvimento de um projeto de *software* independente do paradigma; porém, sua aplicabilidade maior é no paradigma orientado a objetos. Os padrões de projetos facilitam a reutilização de soluções e arquiteturas de *software* de forma fácil e flexível, reduzindo a complexidade do projeto e resolvendo os problemas apresentados.

Neste capítulo, você vai aprender a identificar padrões de projetos da gangue dos quatro, do paradigma orientado a objetos. Você também

vai verificar como escolher, dentre os 23 padrões desse segmento, o melhor padrão para a aplicação em questão. Por fim, você vai analisar as vantagens de utilizar cada tipo de padrão de projetos.

Definição e surgimento dos padrões de projetos

Os **padrões de projetos**, comumente conhecidos como *design patterns*, são modelos, isto é, referências aplicadas ao projeto, trazendo soluções para problemas específicos do desenvolvimento do projeto de *software* orientado a objetos. Os padrões são aplicáveis a outros paradigmas, porém, não são tão relevantes quanto no paradigma orientado.

Por meio dos *design patterns*, é possível definir nomes e descrever o problema e a solução a ser aplicada ao projeto. Eles trazem exemplos de implementações e uma organização geral de classes e objetos. Os padrões de projetos facilitam a reutilização de soluções e arquiteturas bem-sucedidas na construção de um projeto com **paradigma orientado a objetos**. Além disso, eles promovem a organização no código e garantem ao programador um código limpo e padronizado, além de melhorar a documentação e a manutenção do sistema, ao fornecer uma especificação de interações entre os objetos das classes.

Em resumo, os padrões de projetos auxiliam o projetista, o desenvolvedor e o analista a obterem um projeto adequado, sendo uma solução para um problema em um determinado **contexto**.



Fique atento

- **Contexto:** conjunto de situações que se repetem, nas quais o *design pattern* pode ser aplicado.
- **Problema:** conjunto de situações — objetivos e limitações — que ocorrem dentro do contexto.
- **Solução:** estrutura formal a ser aplicada na resolução do problema.

Os problemas surgem com o desenvolvimento do projeto, podendo ser uma dificuldade em desenvolver ou apenas um desafio. O contexto determina qual padrão deve ser aplicado para a resolução do problema. A solução surge com a escolha do padrão (*pattern*) adequado ao projeto em questão.

Todo padrão de projeto possui elementos ou componentes essenciais, conforme listado a seguir.

- **Nome:** descreve a essência do padrão; é uma referência para poder escrever o problema do projeto.
- **Objetivo:** descreve como o padrão atua, em que modelo de *software* se aplica.
- **Problema:** descreve o problema, em que situação se aplica o padrão.
- **Contexto:** descreve uma situação que complementa o problema.
- **Solução:** descreve a solução; é uma descrição abstrata de um problema e de como as classes e os objetos o resolverão.
- **Consequências:** descreve as vantagens e desvantagens da utilização do padrão; são críticas para a avaliação de alternativas de projetos e a compreensão de custos e benefícios para aplicação no *software* desenvolvido.

O padrão de projetos surgiu como um conceito de arquitetura, no final dos anos 1970, documentado no livro *A Pattern Language: Towns, Buildings, Construction*. Nesse livro, os arquitetos Christopher Alexander, Sara Ishikawa e Murray Silverstein catalogaram 253 tipos de problemas de projeto e desafios e analisaram cada situação, descrevendo e propondo um padrão de solução. O livro foi publicado em português em 2012, com o nome *Uma Linguagem de Padrões — a Pattern Language*.

Nos anos 1980, os engenheiros de *software* Kent Beck e Ward Cunningham começaram a aplicar a ideia de padrões na área de programação em uma palestra denominada "Utilizando a linguagem dos padrões para programação orientada a objetos", em tradução livre. Porém, somente nos anos 1990, os padrões ganharam credibilidade, após serem publicados no livro *Design Patterns — Elements of Reusable Object-Oriented Software*, escrito por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides.

O livro apresentou 23 padrões de projetos. Os autores ficaram conhecidos como a gangue dos quatro (GoF, do inglês *gang of four*), termo utilizado nos padrões definidos no livro.

Os padrões facilitam o desenvolvimento do projeto, conforme apontam Gamma *et al.* (2000, p. 17):

Uma coisa que os melhores projetistas sabem que não devem fazer é resolver cada problema a partir de princípios elementares ou do zero. Quando encontram uma boa solução, eles a utilizam repetidamente. Consequentemente, você encontrará padrões de classes e de comunicação entre objetos. Esses padrões resolvem problemas específicos de projetos e tornam os projetos orientados a objetos mais flexíveis e, em última instância, reutilizáveis.

Vantagens em utilizar padrões de projetos

Os padrões de projetos são modelos que já foram utilizados, implementados e testados em diversos projetos, o que traz segurança ao utilizá-los, além de outras vantagens, descritas a seguir.

- Com o uso de padrões de nomenclatura, o projeto possuirá nomes padronizados, trazendo referência e credibilidade para o *software* desenvolvido.
- Melhorar técnicas e princípios da programação orientada a objetos, como implementação de herança, composição e polimorfismo.
- Aumentar a organização e a manutenção de projetos, já que esses padrões se baseiam em baixo acoplamento entre as classes e padronização do código.
- Melhorar o desenvolvimento de projetos tanto individualmente quanto em equipe.
- Implementar hierarquias de herança, diminuindo o tamanho do projeto e aprimorando o mesmo.
- Facilitar a manutenção do projeto e a edição do código.
- Utilizar o padrão para a refatoração de um código ruim, altamente acoplado e de baixa coesão.



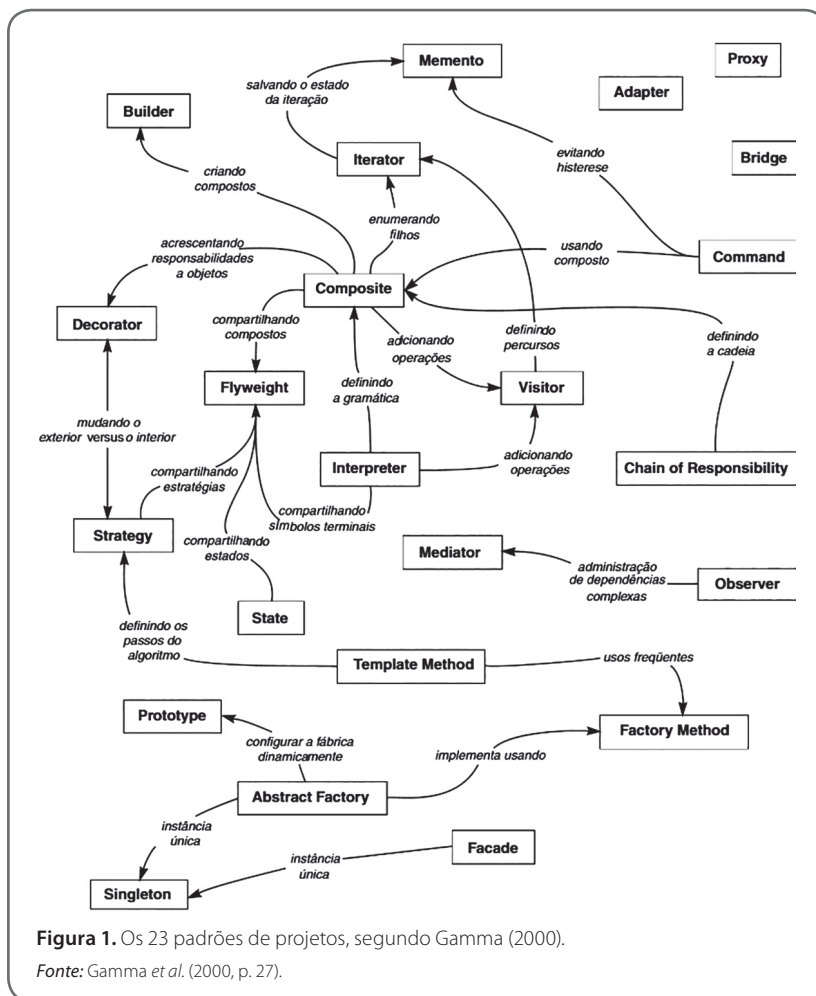
Fique atento

Veja a seguir a terminologia relacionada ao programa orientado a objetos.

- **Classe:** representa um conjunto de objetos com características afins. Uma classe define o comportamento do objeto, por meio de métodos, e quais estados ele é capaz de manter, por meio de atributos.
- **Atributos:** são características de um objeto. Basicamente, trata-se da estrutura de dados que vai representar a classe.
- **Métodos:** definem as habilidades dos objetos. São ações que o objeto pode fazer ou sofrer.
- **Objeto:** instância da classe, elemento que foi gerado por meio da classe.
- **Interface:** é um contrato entre a classe e o mundo externo. Quando uma classe implementa uma interface, ela está comprometida a fornecer o comportamento publicado pela interface.
- **Abstração:** é a habilidade de se concentrar nos aspectos essenciais de um contexto qualquer, ignorando características menos importantes ou acidentais. Em modelagem orientada a objetos, uma classe é uma abstração de entidades existentes no domínio do sistema de *software*.
- **Associação:** é o mecanismo pelo qual um objeto utiliza os recursos de outro. Pode tratar-se de uma associação simples "usa um" ou de um acoplamento "parte de".
- **Coesão:** é quando as tarefas que um elemento realiza estão relacionadas a um mesmo conceito; é o princípio da responsabilidade única, isto é, a ideia de que a classe não deve assumir responsabilidades que não são suas.
- **Sobrecarga:** é a utilização do mesmo nome para símbolos ou métodos com operações ou funcionalidades distintas. Geralmente os métodos se diferenciam pela sua assinatura.
- **Acoplamento:** significa o quanto uma classe depende da outra para funcionar. Surge da associação entre as classes; conforme for a dependência entre ambas, surge o termo "fortemente acopladas".
- **Refatoração:** é o processo de alterar um *software* de maneira que não mude o seu comportamento externo e ainda melhore a sua estrutura interna, modificando apenas a estrutura interna do código.

Qual é o melhor padrão de projeto para o seu *software*?

Encontrar o padrão de projeto adequado ao *software* é difícil. Os padrões de GoF, segundo Gamma *et al.* (2000), apontam 23 padrões na lista de escolhas. Porém, é fundamental seguir alguns critérios, com o objetivo de solucionar o problema do projeto sem cometer erros e tornando o projeto o mais padronizado e organizado possível. Observe a Figura 1.



Cada padrão de projeto é dividido em seções, consistindo em critérios que apresentam e justificam seu uso, possibilitando comparações, entendimento e aplicabilidade. Entre os critérios para a escolha do melhor padrão, destacam-se os seguintes:

1. Considerar como os padrões de projeto solucionam problemas de projeto. Avaliar a necessidade do *software* e qual é a intenção ao utilizar o padrão.
2. Examinar qual é a intenção do padrão, o que faz, de fato, o padrão de projeto, quais são os princípios do modelo e se o mesmo solucionará o problema em questão.

3. Estudar como os padrões se relacionam. Avaliar, por meio da tabela de escopo e propósitos do projeto (Quadro 1), qual é a relação existente entre os padrões.
4. Estudar as semelhanças existentes entre os padrões. Verificar semelhanças entre os grupos de padrões de criação, estruturados e comportamentais.
5. Examinar uma causa de reformulação de projeto.
6. Considerar o que deveria ser variável no seu projeto; ou seja, em vez de considerar o que pode forçar uma mudança em um projeto, deve-se considerar o que se deseja mudar sem necessitar projetá-lo.

Quadro 1. Escopo e propósitos do projeto

		Propósito		
		De criação	Estrutural	Comportamental
Escopo	Classe	<i>Factory Method</i>	<i>Adapter (class)</i>	■ <i>Interpreter</i> ■ <i>Template Method</i>
	Objeto	■ <i>Abstract Factory</i> ■ <i>Builder</i> ■ <i>Prototype</i> ■ <i>Singleton</i>	■ <i>Adapter (object)</i> ■ <i>Bridge</i> ■ <i>Composite</i> ■ <i>Decorator</i> ■ <i>Façade</i> ■ <i>Flyweight</i> ■ <i>Proxy</i>	■ <i>Chain of Responsibility</i> ■ <i>Command</i> ■ <i>Iterator</i> ■ <i>Mediador</i> ■ <i>Memento</i> ■ <i>Observer</i> ■ <i>State</i> ■ <i>Strategy</i> ■ <i>Visitor</i>

Fonte: Adaptado de Gamma et al. (2000).

Analisando o Quadro 1, percebe-se que o primeiro critério abordado é a **finalidade do padrão**, separada em três propósitos: criação, estrutura e comportamento. Além do propósito, há o escopo definido, que estabelece se o padrão é relacionado a classes e subclasses ou envolve objetos. Os padrões relacionados a classes e subclasses são estáticos, enquanto os padrões relacionados a objetos são dinâmicos, isto é, podem ser modificados no tempo real do projeto.

Tipos de padrões de projetos

Seguindo o repositório GoF definido no livro de Gamma *et al.* (2000), os padrões são listados de acordo com o contexto aplicado e o tipo de problema. Os principais padrões de projetos são divididos em segmentos de criação (*creational patterns*), estrutura (*structural patterns*) e padrões de comportamento (*behavioral patterns*).

Creational patterns (padrões de criação)

O objetivo desse tipo de padrão é a abstração da instância de objetos. É possível criar um objeto sem se preocupar com o todo envolvido na criação desse componente. Esse padrão abstrai ou adia o processo de criação, tornando o sistema independente de como os seus objetos são criados. O mesmo pode impedir que um sistema crie mais de um objeto de uma classe, ou pode postergá-lo até o tempo de execução.



Exemplo

Suponha o projeto do jogo Imagem e Ação, em que os jogadores desenharam objetos para que os outros adivinhem. O usuário pode desenhar qualquer forma, sendo figuras vetoriais geométricas ou vetoriais à mão livre. Supondo que cada forma no programa de desenho seja representada por um objeto, no tempo de execução do projeto, não se sabe a forma que o usuário desenhará. Com base na entrada do jogador, o programa deve ser capaz de determinar em que classe instanciar um objeto. Se o jogador criar um retângulo na interface gráfica do utilizador (GUI, do inglês *graphical user interface*), nosso programa deve estar preparado para a instância de um objeto da classe `Rect()`. Quando o jogador decide qual objeto desenhar, o programa deve determinar em que subclasse específica deve instanciar esse objeto.

Os padrões de criação são os seguintes.

- **Abstract Factory:** fornece uma interface para a criação de conjuntos de objetos relacionados ou dependentes, sem precisar especificar sua classe.
- **Builder:** separa a construção de um objeto complexo da sua representação. Possibilita que seu processo de construção crie diferentes representações.

- **Factory Method:** define a interface para a criação de um objeto, mas são as subclasses que decidem qual classe deve ser instanciada.
- **Prototype:** especifica os tipos de objetos a serem criados usando uma instância prototípica e criando objetos copiando esse protótipo.
- **Singleton:** garante que a classe tenha somente uma instância e fornece um ponto global de acesso a ela.

Structural patterns (padrões estruturais)

O objetivo desse padrão é a organização e a estruturação das classes e dos seus relacionamentos com os objetos. Ao contrário do padrão de criação, o padrão de estrutura é voltado para os objetos, descrevendo maneiras de montá-los, isto é, a maneira como as classes e os objetos serão compostos para formar estruturas maiores. A flexibilidade obtida pela composição de objetos provém da capacidade de mudar a composição em tempo de execução.



Exemplo

Segundo Ricardo (2006), do site DevMedia, um exemplo do uso do padrão estruturado é no *framework* Hibernate. Este faz uso do padrão *proxy* ao executar a técnica utilizada para acessar o banco de dados apenas quando for necessário. Em muitas buscas, o Hibernate usa o método `session.load(id)`; nesse momento, um *proxy* para o objeto real é retornado. Nesse caso, o objeto ainda não está completamente preenchido, pois nenhum SQL foi realizado até esse momento. Apenas quando uma propriedade desse objeto (método `getX()`) ou um relacionamento (por exemplo, `empresa.getFuncionarios()`) for chamado, a consulta no banco será realizada — tudo isso de forma transparente para o cliente.

Os padrões de estrutura são os seguintes.

- **Adapter:** converte a interface de uma classe em outra interface esperada pelo cliente. O *Adapter* permite que as classes trabalhem em conjunto para promover essa mudança.
- **Bridge:** separa uma abstração da sua implementação, permitindo uma variação individual de cada uma.
- **Composite:** compõe objetos em estruturas hierárquicas, como árvores. Trata objetos individuais e composições de objetos de forma uniforme.

- **Decorator:** atribui de forma dinâmica responsabilidades adicionais a um objeto.
- **Facade:** fornece uma interface unificada para um conjunto de interfaces em subsistema.
- **Flyweight:** suporta grandes quantidades de objetos de forma eficiente, por meio de compartilhamento.
- **Proxy:** fornece um objeto representante (*surrogate*) ou marcador de outro objeto.

Behavioral patterns (padrões comportamentais)

O objetivo é delegar responsabilidades, definindo como os objetos devem se comportar e se comunicar. Esses padrões se concentram nos algoritmos.



Exemplo

Os elementos GUI utilizam padrões comportamentais para registrar eventos de interação com componentes, como botões, registrando mudanças de *status* — por exemplo: pressionado, acessado. O padrão também permite que um sistema salve o estado de um objeto, de modo que esse estado possa ser restaurado — por exemplo, a função desfazer.

São 11 os padrões comportamentais, que são descritos a seguir.

- **Chain of Responsibility:** evita o acoplamento de remetente de uma solicitação ao destinatário, dando chance ao objeto para tratar a solicitação.
- **Command:** encapsula uma solicitação como objeto, permitindo parametrizar clientes com diferentes solicitações, enfileirando ou registrando os “logs” de solicitações.
- **Interpreter:** define para a linguagem uma representação gramatical dentro do interpretador.
- **Iterator:** possibilita o acesso sequencial dos elementos de uma agregação de objetos, sem a necessidade de expor sua representação subjacente.
- **Mediator:** define um objeto que encapsula a forma como um conjunto de objetos interage.
- **Memento:** captura e externaliza um estado interno do objeto sem violar seu encapsulamento, possibilitando restaurar para esse estado — por exemplo, um controle de históricos de ações.

- **Observer:** define uma dependência um-para-muitos entre objetos; quando um objeto muda, todos os seus dependentes são notificados e atualizados automaticamente.
- **State:** permite que o objeto altere seu comportamento quando o estado interno for modificado.
- **Strategy:** define uma família de algoritmos, encapsulando cada um e os tornando intercambiáveis.
- **Template Method:** define o esqueleto de um algoritmo em uma operação, postergando a definição de alguns passos para subclasses.
- **Visitor:** representa uma operação a ser executada sobre os elementos da estrutura de um objeto.



Link

Conhecer os padrões possibilita desenvolver soluções cada vez melhores e mais eficientes, que podem ser reutilizadas. O uso de padrões de projeto propicia a construção de aplicações e ou estruturas de código de forma flexível e a documentação de soluções reaproveitáveis. Leia mais sobre os padrões de projetos e seus objetivos acessando o *link* a seguir.

<https://qrqo.page.link/dHfTZ>



Referências

GAMMA, E. *et al.* *Padrões de projeto: soluções reutilizáveis de software orientado a objetos*. 1. ed. Porto Alegre: Bookman, 2000.

RICARDO. Conheça o Pattern Proxy — GoF (Gang of Four). *Devmedia*, Rio de Janeiro, 2006. Disponível em: <https://www.devmedia.com.br/conheca-o-pattern-proxy-gof-gang-of-four/4066>. Acesso em: 17 jun. 2019.

Leituras recomendadas

ALEXANDER, C. *et al.* *Uma linguagem de padrões*. Porto Alegre: Bookman, 2012.

FOWLER, M. *Padrões de arquitetura de aplicações corporativas*. Porto Alegre: Bookman, 2007.

HORSTMANN, C. *Padrões e projetos orientados a objetos*. 2. ed. Porto Alegre: Bookman, 2007.

LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientado a objetos e ao desenvolvimento iterativo*. 3. ed. Porto Alegre: Bookman, 2007.

METSKER, S. J. *Padrões de projeto em Java*. Porto Alegre: Bookman, 2004.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.

SHALLOWY, A. *Explicando padrões de projeto: uma nova perspectiva em projeto orientado a objetos*. Porto Alegre: Bookman, 2004.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Dica do Professor

A utilização de padrões de projeto traz uma série de benefícios no desenvolvimento de *software* orientado a objetos. Esses padrões facilitam a reutilização de soluções e arquiteturas, tornam o *software* mais flexível, reduzem a complexidade do projeto, padronizam a linguagem de programação utilizada e favorecem a manutenção do código ao longo do tempo.

No entanto, diante de mais de vinte padrões disponíveis para escolha, encontrar o padrão de projeto adequado ao *software* pode ser um desafio. É fundamental seguir critérios bem definidos para solucionar os problemas do projeto de forma eficiente, evitando erros e garantindo que o *software* seja desenvolvido de maneira padronizada e organizada.

Nesta Dica do Professor, você vai analisar os critérios que justificam o uso de um padrão de projeto, tendo como exemplo o padrão Singleton, que faz parte do grupo de padrões de criação. Você vai, então, compreender a importância de selecionar cuidadosamente o padrão de projeto mais adequado para cada contexto, visando a obter os melhores resultados no desenvolvimento do *software*.

As imagens do vídeo a seguir possuem audiodescrição. Para acessar o recurso,

[clique aqui](#)



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Exercícios

1) Os padrões de projeto são divididos em três grupos:

- **CREATIONAL PATTERNS**: padrões de criação, cujo objetivo é a abstração da instância de objetos.
- **STRUCTURAL PATTERNS**: padrão estrutural, cujos objetivos são a organização e a estrutura das classes, bem como o relacionamento destas com os objetos.
- **BEHAVIORAL PATTERNS**: padrão comportamental, cujo objetivo é delegar responsabilidade, definindo como os objetos devem se comportar e se comunicar.

Nesse contexto, analise as seguintes afirmações:

I. O objetivo do padrão de projeto Singleton, do grupo Padrão de Criação, é conter um único objeto na classe.

II. O padrão de projeto Proxy, do grupo de padrões estruturais, permite que o sistema utilize um objeto no lugar de outro. Ele é muito utilizado em processo de carregamento de imagens, por exemplo.

III. O padrão de projeto Memento, do grupo de padrões estruturais, permite guardar e restaurar um estado do objeto, sendo muito utilizado para manipular históricos.

Qual(is) delas é(são) verdadeira(s)?

- A) Somente III.
- B) Somente II.
- C) I e III.
- D) I e II.
- E) I, II e III.

2) Os padrões de projeto facilitam a reutilização de soluções e arquiteturas de *softwares* orientados a objetos de forma flexível e fácil, reduzindo a complexidade do projeto e resolvendo os problemas.

Considerando os padrões da obra *Padrões de Projeto – Soluções reutilizáveis de software orientado a objetos*, de Erich Gamma, assinale a alternativa correta:

- A) O GoF, composto por 23 padrões, tornou-se base para outros padrões.
- B) Os padrões GoF estão divididos em categorias para projetos de criação, projetos de estrutura e projetos de transação.
- C) Dois exemplos de padrões GoF são Composite e Decorator, os quais se encaixam na categoria de padrões de criação.
- D) Os padrões GoF melhoram as técnicas e os princípios da programação orientada a objetos e estruturada.
- E) Os padrões GoF são utilizados unicamente para refatoração de códigos ruins, altamente acoplados e de baixa coesão.

- 3) O objetivo do padrão de criação é a abstração da instância de objetos, sendo possível criar um objeto sem se preocupar com o todo envolvido na criação desse componente. Dessa forma, esse padrão abstrai ou adia o processo de criação, tornando o sistema independente de como seus objetos são criados.

Avalie as afirmações a seguir sobre o padrão de projeto Singleton, do grupo de projetos de criação:

I. O padrão Singleton pode ser usado para determinar uma quantidade fixa de instâncias de uma classe que podem ser criadas em uma aplicação.

II. A classe que implementa o padrão Singleton deve manter públicos seus construtores para facilitar a instância de classes.

III. O padrão Singleton é implementado por meio de atributos e métodos estáticos na classe em que é aplicado, o que permite uma instância da classe e um ponto global de acesso a ela.

Qual(is) delas é(são) verdadeira(s)?

- A) I, II e III.
 - B) I e II.
 - C) I e III.
 - D) Somente I.
 - E) II e III.
- 4) Os padrões de projeto facilitam a reutilização de soluções e arquiteturas bem-sucedidas na construção de um projeto com paradigma orientado a objetos (POO), possibilitando trazer

ao código organização e, ao programador, habilidade, experiência e um código limpo e padronizado.

Como escolher o melhor padrão a ser utilizado?

- A)** A decisão de utilizar o padrão X ou Y depende de quantos desenvolvedores estão envolvidos no projeto, do problema encontrado e de quantas classes o projeto dispõe.
- B)** A decisão de utilizar o padrão X ou Y depende unicamente da experiência do desenvolvedor envolvido no projeto.
- C)** A decisão de utilizar o padrão X ou Y depende do número de classes que o projeto dispõe e se ele apresenta técnicas de polimorfismo e de herança.
- D)** A decisão de utilizar o padrão X ou Y depende da avaliação de alguns critérios como: intenção e objetivo do padrão, motivação, semelhanças dos padrões, estrutura do projeto e aplicabilidade.
- E)** A decisão de utilizar o padrão X ou Y depende do escopo do projeto, se a implementação será nas classes ou objetos e se haverá refatoração e hierarquias de heranças.

5)

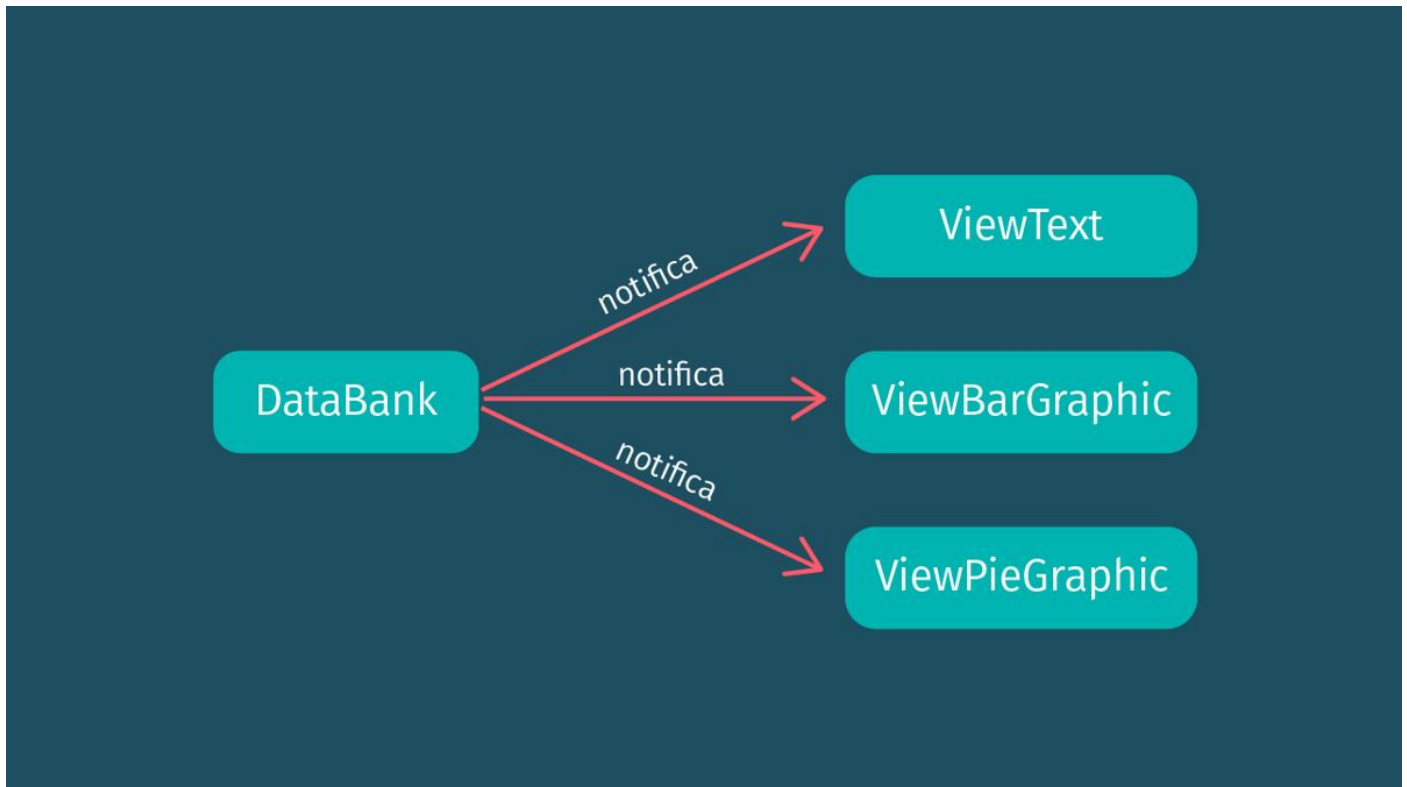
Um projeto desenvolvido para uma agência bancária contém uma classe **DataBank** para armazenar dados relacionados às instruções do banco e mais três classes que exibem dados bancários: **ViewText**, **ViewBarGraphic** e **ViewPieGraphic**.

Cada interação no objeto da classe **DataBank** deve notificar e atualizar os objetos das demais classes.

Ao interagir com o objeto **DataBank**, fazendo uma transação bancária, o objeto da classe **ViewText** deve apresentar um texto descrevendo a transação, enquanto os objetos da classe **ViewBarGraphic** e da **ViewPieGraphic** precisam apresentar graficamente a transação realizada.

A imagem a seguir possui audiodescrição. Para acessar o recurso,

[clique aqui](#)



Analisando o *case* do projeto, qual padrão seria adequado na implementação da atualização de classes, priorizando a organização e a melhoria do sistema proposto?

- A) O mais indicado seria o **Singleton**, padrão de criação, pois trata da instanciação de objetos e comunicação. Quando se instancia a classe `DataBank`, as demais classes são comunicadas e instanciadas.
- B) O mais indicado seria o **Observer**, padrão comportamental, pois ele possibilita a dependência um-para-muitos entre objetos. Quando o objeto da classe `DataBank` muda, todos os objetos das demais classes são notificados e atualizados automaticamente.
- C) O mais indicado seria o **Factory Method**, padrão de criação, pois implementa a interface para a criação de um objeto da conta bancária `DataBank` e das subclasses `ViewText`, `ViewBarGraphic` e `ViewPieGraphic`.
- D) O mais indicado seria o **Composite**, padrão estrutural, já que envolve a estrutura do projeto, trabalhando com objetos de forma hierárquica, sendo `DataBank` classe "pai" e demais classes "filhas".
- E) O mais indicado seria o **Memento**, padrão comportamental, pois captura e externaliza a ação da classe `DataBank`, possibilitando restaurar o estado das subclasses.

Na prática

Na área da arquitetura de *software*, os padrões de projeto desempenham um papel fundamental ao fornecer soluções testadas e comprovadas para desafios recorrentes. Esses padrões são categorizados em três segmentos essenciais: criação, estrutura e comportamento. Cada segmento aborda problemas específicos dentro de um domínio, fornecendo diretrizes valiosas para a criação de códigos e diagramas eficientes.

O padrão de criação, em particular, concentra-se na abstração da instância de objetos, permitindo soluções flexíveis e reutilizáveis. Nos sistemas de desenvolvimento e *software*, a eficiência e a segurança são aspectos essenciais a serem considerados. Uma abordagem que se destaca para lidar com a criação única de instâncias de objetos é o padrão de projeto Singleton. Esse padrão pertence à categoria de padrões de criação, que visam à estruturação otimizada das instâncias de classes.

No contexto da programação orientada a objetos, compreender e aplicar o padrão Singleton é uma habilidade essencial para garantir a segurança, a consistência e a eficiência de sistemas que requerem apenas uma única instância de determinada classe.

Confira, neste Na Prática, o exemplo da implementação da classe de configuração com o uso do padrão de criação Singleton.

A imagem a seguir possui audiodescrição. Para acessar o recurso,

[clique aqui](#)

IMPLEMENTAÇÃO DA CLASSE DE CONFIGURAÇÃO COM O USO DO PADRÃO DE CRIAÇÃO SINGLETON

1. IMPLEMENTAÇÃO DA CLASSE APRESENTADA NO CASE:

Linguagem utilizada: Java

```
1 public class Configuracao {
2
3     //Atributo para guardar uma única instância da classe - uso padrão
4     private static Configuracao instancia;
5     //Atributos privados:
6     private String ip;
7     private int porta;
8     private String usuario;
9     private String senha;
10
11     //Método Construtor PRIVADO:
12     private Configuracao(){
13     }
14
15     //Método estático que retorna a instância, e se não existir
16     instância a própria classe:
17     public static Configuracao getInstancia(){
18         if (instancia == null){
19             instancia = new Configuracao();
20         }
21         return instancia;
22     }
23
24     public void autenticar() {
25     }
26     /*
27     * Implementação dos métodos acessores Getters e Setters e outros
28     métodos
29     */
30 }
```

Figura 1 - Classe Configuração Implementação em Java do case apresentado no Na Prática

2. NOTE O USO DO ATRIBUTO ESTÁTICO INSTÂNCIA:

```
3 //Atributo para guardar uma única instância da classe - uso padrão
4 private static Configuracao instancia;
```

Figura 2 - Atributo instância

Este atributo guardará uma única instância desta classe.

3. CONSTRUTOR PRIVADO PARA GARANTIR QUE NINGUÉM PODERÁ ACESSÁ-LO E, DESSA FORMA, INSTANCIAR FORA DA CLASSE.

```
11 //Método Construtor PRIVADO:
12 private Configuracao(){
13 }
```

Figura 3 - Método para construir um objeto

No exemplo, o método construtor encontra-se em estado *default*, vazio, porém ele pode ser implementado com parâmetros.

O que será visível fora do escopo são somente os métodos e atributos declarados como **public**.

Os métodos e atributos declarados como **private** estão acessíveis somente dentro do seu escopo, dentro da sua classe.

4. POR FIM, A IMPLEMENTAÇÃO DO MÉTODO GETINSTANCIA:

```
15 //Método estático que retorna a instância, e se não existir instância
16 a própria classe:
17 public static Configuracao getInstancia(){
18     if (instancia == null){
19         instancia = new Configuracao();
20     }
21     return instancia;
22 }
```

Figura 4 - Método com implementação do padrão Singleton

Método estático que retorna a instância e, se não existir, ele instancia a própria classe. Ao executar o método apresentado na linha 16, o sistema verificará se já existe uma instância rodando e, se sim, retorna tal instância, caso contrário ele a cria. Dessa forma, também se tem o controle que ela foi criada uma **única vez**.

APÓS A CRIAÇÃO DA CLASSE DE CONFIGURAÇÃO, PODE-SE CHAMAR A INSTÂNCIA EM QUANTAS CLASSES DESEJAR, UMA VEZ QUE SEMPRE HAVERÁ A CERTEZA DE EXISTIR APENAS UMA.

```
1 Configuracao conf1 = Configuracao.getInstancia () ;
2
3 conf1 . autenticar () ;
```

Figura 5 – Invocando o método responsável pelo controle da instância da classe Configuração

A instância só poderá ser feita por meio do método `getInstancia()`.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Às vezes, há necessidade de se ter apenas uma instância de determinada classe do sistema, classes de configuração, janelas únicas e classes com atributos únicos, modelos de sistemas que podem utilizar o padrão de projeto do seguimento criação, isto é, o padrão Singleton.

O objetivo do padrão Singleton é assegurar que uma classe tenha uma única instância, provendo a ela um ponto de acesso global. No exemplo, Eduarda fez uso do padrão Singleton, implementando no seu código o método `getInstance()`.

Saiba mais

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

Padrão de projeto Strategy

Neste artigo, você vai saber mais sobre o Strategy, um padrão comportamental de projeto que permite a mudança de comportamento de um conjunto de classes em tempo de execução, fazendo com que seus objetos operem de forma diferente dependendo da operação a ser realizada.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Avaliação de *code smells* e padrões de projeto em aplicações com Angular e React

As tecnologias para desenvolvimento *web* Angular e React são muito populares atualmente. Neste artigo, você vai conferir uma análise de quinze aplicações desenvolvidas com Angular e quinze com React, além de analisar a presença de *code smells* e padrões de projeto nesses programas.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Padrão de requisitos no ciclo de vida de *software*: um mapeamento sistemático

Pesquisas demonstram que a prática de reúso por meio de padrões de requisitos é uma alternativa eficaz para tratar problemas de qualidade de especificação, com o benefício adicional de economia de tempo. Neste artigo, você vai conferir mais sobre esse tema.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.